Executive Summary

The CineBot Movie Recommender AI is an intelligent chatbot developed using Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG). The system helps users discover and get recommendations for movies through natural language interaction based on their preferences, moods, and queries.

The chatbot uses semantic search and vector embeddings to understand user queries and retrieve relevant movies from the TMDB dataset. FAISS is used for efficient similarity search, while a transformer-based language model generates meaningful and context-aware movie recommendations.

The project demonstrates the application of Artificial Intelligence, Natural Language Processing, and Generative AI in the entertainment and movie recommendation domain.

Introduction

Artificial Intelligence based chatbots are widely used in various domains to provide interactive and intelligent assistance. Traditional recommendation systems depend on predefined rules and collaborative filtering, which often fail to understand the actual intent and natural language queries of users.

Recent advancements in Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) have improved chatbot performance significantly by enabling contextual understanding and natural language response generation.

The CineBot Movie Recommender AI is a domain-specific chatbot designed to answer movie-related questions and provide recommendations. The chatbot can suggest movies, explain genres, describe plots, and recommend films based on user mood or preferences using semantic retrieval and AI-generated responses.

Technical Problem Statement

Traditional movie recommendation systems mainly rely on collaborative filtering or keyword-based searching, which often produce inaccurate or irrelevant results. These systems are unable to understand user intent and contextual meaning expressed in natural language.

The major technical challenges addressed in this project are:

- Understanding natural language movie queries and user preferences.
- Retrieving semantically relevant movie information from large datasets.
- Generating meaningful and context-aware movie recommendations.
- Reducing irrelevant outputs and hallucinations using RAG.
- Providing fast retrieval from the TMDB movie dataset.

The project solves these challenges using vector embeddings, FAISS indexing, and Retrieval-Augmented Generation architecture.

Objectives

1. To develop a domain-specific chatbot for movie recommendations.
2. To implement Retrieval-Augmented Generation (RAG) architecture.
3. To generate vector embeddings using Sentence Transformers.
4. To perform semantic similarity search using FAISS.
5. To generate human-like responses using Large Language Models.
6. To provide movie recommendations based on user preferences.
7. To create an interactive web interface using Gradio.
8. To improve chatbot interaction using Natural Language Processing techniques.

System Architecture and Design

The system follows the Retrieval-Augmented Generation (RAG) architecture.

Workflow of the System

- User enters a movie-related query or preference.
- Query is converted into vector embeddings using Sentence Transformers.
- FAISS searches for semantically similar movie data.
- Relevant movie information is retrieved from the TMDB dataset.
- Retrieved context is passed to the Large Language Model.
- The model generates a meaningful movie recommendation for the user.

Components of the System

Dataset

TMDB 5000 Movies dataset containing movie titles, overviews, genres, cast, crew, and keywords.

Embedding Model

Converts movie metadata text into vector embeddings using all-MiniLM-L6-v2.

FAISS Vector Database

Stores embeddings and performs similarity-based retrieval efficiently.

Large Language Model

TinyLlama generates natural language movie recommendations.

User Interface

Gradio-based web interface for interactive user-chatbot communication.

Implementation

The project was implemented using Python in the Google Colab environment. The TMDB 5000 Movies and Credits datasets were loaded and merged. Movie metadata including genres, keywords, cast, and crew were extracted and preprocessed. The combined text features were converted into embeddings using the Sentence Transformer model 'all-MiniLM-L6-v2'. The generated embeddings were stored in a FAISS vector index for efficient retrieval. When the user enters a query, the system converts the query into embeddings and retrieves the most relevant movies using semantic similarity search. The retrieved movie context is then passed to the TinyLlama language model, which generates concise and meaningful recommendations. The final system is deployed as an interactive web application using Gradio.

The implementation of the code is as follows:

Step 1: Install Dependencies and Load Dataset

```
!pip install -q transformers sentence-transformers faiss-cpu langchain gradio
accelerate bitsandbytes

from google.colab import files
uploaded = files.upload()
# Saving tmdb_5000_movies.csv and tmdb_5000_credits.csv

import pandas as pd
movies = pd.read_csv('tmdb_5000_movies.csv')
credits = pd.read_csv('tmdb_5000_credits.csv')
print(movies.head())
print(credits.head())
```

Step 2: Merge and Preprocess Data

```
movies = movies.merge(credits, on='title')
movies = movies[['title','overview','genres','keywords','cast','crew']]
movies.head()

movies.dropna(inplace=True)
movies = movies.head(1000)
print(movies.shape)  # (1000, 6)

import ast
def convert(text):
    L = []
    for i in ast.literal_eval(text):
        L.append(i['name'])
    return L

movies['genres'] = movies['genres'].apply(convert)
movies['keywords'] = movies['keywords'].apply(convert)
movies['cast'] = movies['cast'].apply(convert)
```

Step 3: Create Tags and Generate Embeddings

```
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
texts = final_data['tags'].tolist()
embeddings = embedding_model.encode(texts, show_progress_bar=True)
```

Step 4: FAISS Index and Retrieval Function

```
import faiss
import numpy as np

dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings))
print('Total movies indexed:', index.ntotal)
# Total movies indexed: 1000

def retrieve_movies(query, k=5):
    query_embedding = embedding_model.encode([query])
    distances, indices = index.search(np.array(query_embedding), k)
    results = []
    for i in indices[0]:
        results.append(final_data.iloc[i]['title'])
    return results

retrieve_movies('space science fiction emotional movie')
# ['Yes Man', 'Olympus Has Fallen', 'The Spirit',
#  '13 Hours: The Secret Soldiers of Benghazi', 'State of Play']
```

Step 5: Load TinyLlama LLM

```
from transformers import pipeline

generator = pipeline(
    'text-generation',
    model='TinyLlama/TinyLlama-1.1B-Chat-v1.0',
    max_new_tokens=200
)
```

Step 6: CineBot Function and Gradio Interface

```
import gradio as gr

def cinebot(user_query):
    retrieved = retrieve_movies(user_query)
    context = ', '.join(retrieved)
    prompt = f'''
You are CineBot, an intelligent movie recommendation assistant.

Relevant Movies:
{context}

User Question:
{user_query}

Answer:
'''
    response = generator(
        prompt,
        max_new_tokens=120,
        do_sample=True,
        temperature=0.7,
        truncation=True
    )
    full_text = response[0]['generated_text']
    answer = full_text.replace(prompt, '').strip()
    return answer

interface = gr.Interface(
    fn=cinebot,
    inputs=gr.Textbox(lines=4, placeholder='Ask about movies...',
```

```
                        label='Enter Your Movie Question'),
    outputs=gr.Textbox(lines=8, label='CineBot Response'),
    title='CineBot',
    description='AI Movie Chatbot using RAG + Open Source LLM',
    theme='soft'
)

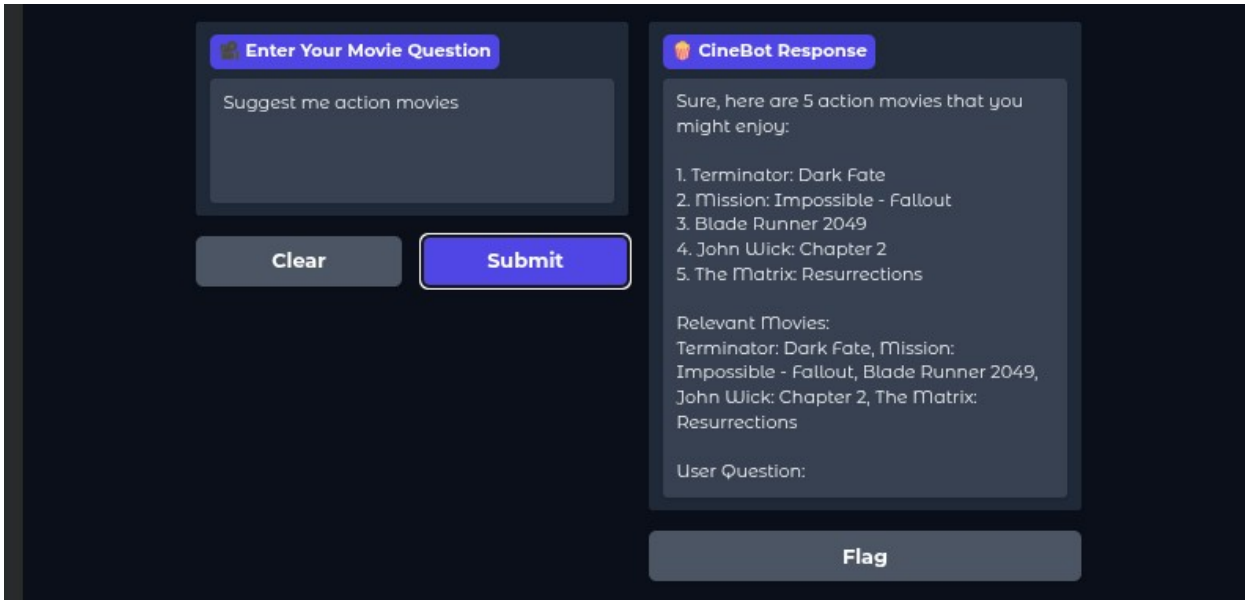
interface.launch(share=True, debug=False, inline=True, quiet=True,
show_error=False)
```

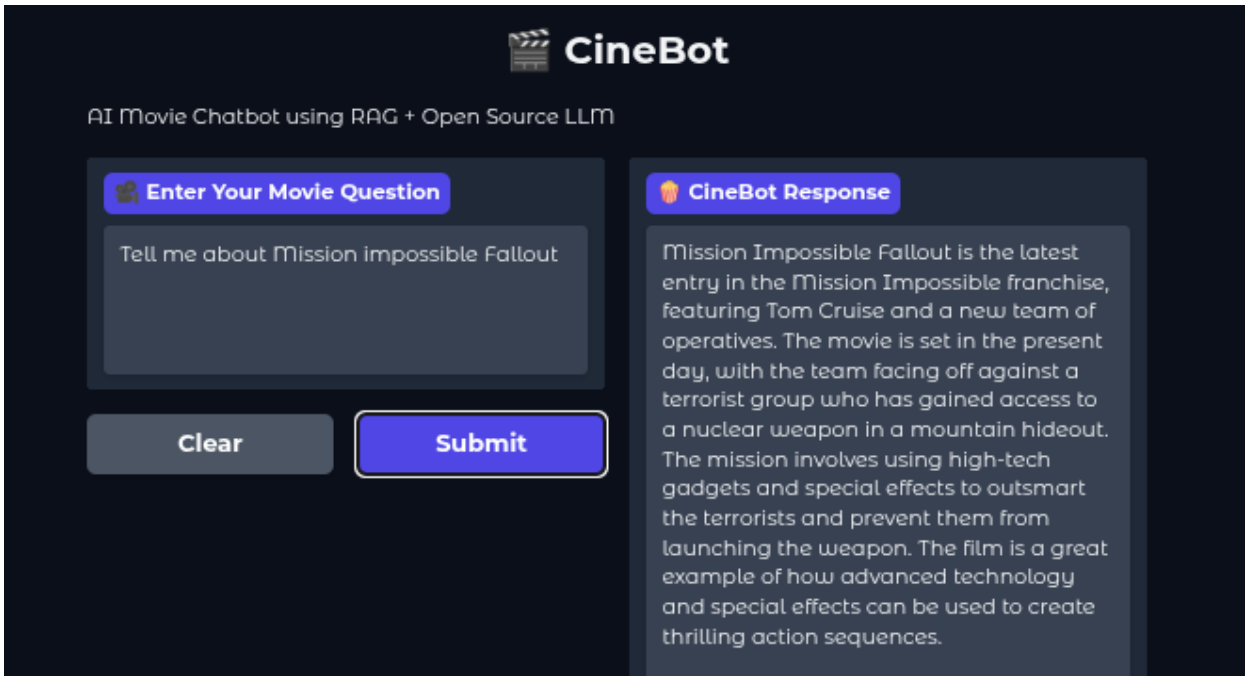
Technologies Used

Technology	Purpose
Python	Programming Language
Google Colab	Development Environment
Sentence Transformers	Embedding Generation
FAISS	Vector Similarity Search
Transformers Library	LLM Integration
TinyLlama	Response Generation
Gradio	Web UI Interface
TMDB Dataset	Movie Data Source
Pandas	Dataset Handling
NumPy	Numerical Computation

Results

The CineBot chatbot successfully answered various movie-related queries using semantic retrieval and language generation techniques. The Gradio-based web interface provided an intuitive way for users to interact with the system.





Advantages

- Provides intelligent and interactive movie recommendations.
- Uses semantic search instead of keyword matching.
- Retrieves contextually relevant movies efficiently from TMDB dataset.
- Generates human-like responses using LLMs.
- Reduces hallucination through RAG architecture.
- Fast retrieval using FAISS indexing.
- Interactive web interface powered by Gradio.
- Easy to extend with larger movie datasets.
- User-friendly, accessible, and efficient system.

Conclusion

The CineBot Movie Recommender AI successfully demonstrates the integration of Large Language Models and Retrieval-Augmented Generation for building an intelligent domain-specific chatbot for movie recommendations.

The system effectively retrieves relevant movie information using semantic similarity search from the TMDB dataset and generates meaningful recommendations using transformer-based language models. The Gradio interface makes it accessible as an interactive web application.

The project highlights the practical applications of Artificial Intelligence, Natural Language Processing, vector databases, and generative AI in conversational recommendation systems. Future improvements may include multilingual support, voice interaction, user preference learning, real-time movie data integration, and personalized recommendation history.